

PHASE 5

Project Development Phase

Building the Tableau visualizations and the Flask web application

PulseView – Heart Disease Risk Analysis Dashboard

Project	PulseView – Heart Disease Risk Analysis Dashboard
Document	Project Development Report
Version	1.0
Date	July 2026
Prepared By	Project Team

1. Introduction

This document records how the plan and design were implemented: the development environment, the module-by-module build order, and the key code artifacts that make up the PulseView Flask application.

2. Development Environment Setup

The application is packaged as a standard Python/Flask project with a single dependency.

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
python app.py
The app then runs locally at http://127.0.0.1:5000.
```

requirements.txt pins a single dependency:

```
Flask==3.1.3
```

3. Module-Wise Development

Development followed the six milestones from the Project Planning phase. The table below maps each milestone to what was actually built.

Milestone	What was built
Problem understanding	Problem statement and persona definitions later published on the About page.
Data collection	Heart-disease dataset sourced and connected as a Tableau data source.
Data preparation	Fields cleaned/typed in Tableau (sex, smoking, alcohol, race, diabetes, stroke, outcome, plus age/BMI/cholesterol/blood-pressure fields referenced by personas).
Visualization	Four-panel dashboard and four-scene story authored in Tableau and published to Tableau Public.
Performance testing	Filter behaviour, calculated fields, and rendering speed checked in Tableau (see Phase 6 report).
Web integration	Flask app (app.py), five Jinja2 templates, CSS, and JS built to host and present the embeds.

4. Key Code Artifacts

4.1 Route definitions (app.py)

Five routes back the five pages; the dashboard and story routes pass structured data into their templates:

```
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/dashboard")
def dashboard():
    return render_template(
```

```

        "dashboard.html", panels=DASHBOARD_PANELS,
        embed_url=TABLEAU_DASHBOARD_EMBED_URL,
    )

@app.route("/story")
def story():
    return render_template(
        "story.html", scenes=STORY_SCENES,
        embed_url=TABLEAU_STORY_EMBED_URL,
    )

```

4.2 Data-driven content

Story scenes and dashboard panels are defined as plain Python lists of dictionaries at the top of app.py, so new content never requires a template change:

```

STORY_SCENES = [
    {
        "id": "overview", "label": "Overview",
        "image": "overview.png",
        "title": "Where the risk sits today",
        "text": "...",
    },
    # gender-risk, lifestyle-factors, race-conditions ...
]

DASHBOARD_PANELS = [
    {"title": "Gender vs Heart Disease", "text": "..."},
    # Diabetic vs Stroke, Race-wise, Smoking and Alcohol ...
]

```

4.3 Contact form handling

The contact route validates required fields and uses Flask's flash-messaging for feedback, with the email-send step left as a clearly marked extension point:

```

if not name or not email or not message:
    flash("Please fill in every field before sending.", "error")
    return redirect(url_for("contact"))

# No email backend is wired up yet -- this is where you'd call
# your mail service (e.g. Flask-Mail) with name/email/message.
flash("Thanks, {}! Your message has been received.".format(name), "success")

```

4.4 Story navigation (JavaScript)

Tab clicks, previous/next buttons, and left/right arrow keys all drive the same `goTo(index)` function on the client side, keeping the four scenes in sync:

```

function goTo(index) {
    current = (index + total) % total;
    tabs.forEach(function (tab, i) {
        tab.classList.toggle("active", i === current);
    });
    scenes.forEach(function (scene) {
        scene.classList.toggle("is-active",
            Number(scene.dataset.scene) === current);
    });
}

```

5. Project Directory Structure

heart_disease_flask/

```
■■■ app.py
■■■ requirements.txt
■■■ static/
■   ■■■ css/style.css
■   ■■■ js/main.js
■   ■■■ images/           # dashboard.png + 4 story scene PNGs
■■■ templates/
    ■■■ base.html
    ■■■ index.html
    ■■■ about.html
    ■■■ dashboard.html
    ■■■ story.html
    ■■■ contact.html
```

6. Version Control & Deployment Notes

- A .gitignore is configured at the project root to keep the virtual environment and compiled artifacts out of version control.
- Because the app has no database, deployment only requires a Python 3 runtime, the pinned Flask dependency, and the two Tableau Public embed URLs to be reachable.
- `app.run(debug=True)` is used for local development; a production deployment would swap in a WSGI server (e.g. gunicorn) and disable debug mode.